

**Disciplina:**  
**Arquiteturas de Softwares**

Prof. Esp. Leonardo Dias  
Curso de Sistemas de Informação

# Contêineres

**Dia 09**

Disciplina: Arquiteturas de Softwares

**Arquitetura** não é só “um desenho bonito”, mas sim a estrutura que sustenta o software.

# Contêineres na Arquitetura de Software

## 1. Introdução

Na arquitetura moderna de software, a virtualização de aplicações e serviços tornou-se essencial para garantir **portabilidade, escalabilidade e consistência** entre ambientes.

Os **contêineres** são uma das principais tecnologias que possibilitam isso, permitindo empacotar uma aplicação e todas as suas dependências em uma unidade executável leve e isolada.

## 2. O que são Contêineres

Um **contêiner** é um ambiente isolado que contém:

- O **código da aplicação**
- Suas **dependências**
- Suas **configurações**
- E tudo o que é necessário para executá-la corretamente

Diferente das máquinas virtuais (VMs), os contêineres **compartilham o kernel do sistema operacional** do host, tornando-se **mais leves e rápidos**.

## 3. Diferença entre Contêineres e Máquinas Virtuais

| Aspecto                | Máquina Virtual                      | Contêiner                      |
|------------------------|--------------------------------------|--------------------------------|
| Isolamento             | Total (inclui o sistema operacional) | Parcial (compartilha o kernel) |
| Tempo de inicialização | Lento (vários segundos ou minutos)   | Rápido (frações de segundo)    |
| Tamanho                | Gigabytes                            | Megabytes                      |
| Uso de recursos        | Alto                                 | Baixo                          |
| Portabilidade          | Limitada                             | Alta                           |

|            |                              |                                         |
|------------|------------------------------|-----------------------------------------|
| Ideal para | Infraestruturas tradicionais | Arquiteturas baseadas em microsserviços |
|------------|------------------------------|-----------------------------------------|

**Resumo:** contêineres são mais leves e eficientes, ideais para **ambientes dinâmicos e escaláveis**, como os usados em **DevOps e microsserviços**.

## 4. Arquitetura de um Sistema com Contêineres

Um ambiente baseado em contêineres geralmente possui três camadas principais:

1. **Host OS (Sistema Operacional):** executa o engine de contêineres, como Docker ou containerd.
2. **Engine de Contêineres:** responsável por criar e gerenciar os contêineres.
3. **Contêineres:** instâncias isoladas que executam serviços, aplicações ou partes de um sistema.

## 5. Docker: o Padrão de Fato

O **Docker** é a plataforma mais popular para gerenciamento de contêineres.

Ele permite:

- Empacotar aplicações em **imagens**
- Executá-las em **contêineres**
- Automatizar o ciclo de vida com **Docker Compose** e **Docker Hub**

**Principais conceitos do Docker:**

- **Imagem:** modelo imutável que contém tudo o que o contêiner precisa.
- **Contêiner:** instância em execução de uma imagem.
- **Dockerfile:** script de instruções para construir uma imagem.
- **Registry:** repositório de imagens (ex.: Docker Hub).

## 6. Benefícios dos Contêineres na Arquitetura de Software

1. **Portabilidade:** executam da mesma forma em qualquer ambiente.
2. **Escalabilidade:** permitem subir ou descer instâncias rapidamente.
3. **Isolamento:** cada contêiner roda sem interferir em outros.
4. **Padronização:** ambiente de desenvolvimento idêntico ao de produção.
5. **Eficiência:** uso otimizado de CPU, memória e armazenamento.
6. **Integração com DevOps:** facilitam CI/CD, testes automatizados e deploy contínuo.

## 7. Contêineres e Microsserviços

Os contêineres se encaixam perfeitamente com a **arquitetura de microsserviços**, pois cada serviço pode:

- Ser empacotado e executado de forma independente
- Ser escalado separadamente
- Ter seu próprio ciclo de vida de deploy e atualização

Assim, os contêineres **viabilizam a modularidade e o isolamento** característicos dessa arquitetura.

## 8. Orquestração de Contêineres

Quando há dezenas ou centenas de contêineres, surge a necessidade de **ferramentas de orquestração**.

Essas ferramentas automatizam:

- A distribuição dos contêineres nos servidores
- O balanceamento de carga
- A detecção de falhas
- A escalabilidade automática

◆ **Principais ferramentas de orquestração:**

- **Kubernetes** (padrão de mercado)
- **Docker Swarm**
- **Amazon ECS / EKS**
- **Azure Kubernetes Service (AKS)**

## 9. Desafios e Boas Práticas

**Desafios:**

- Gestão de segurança e segredos
- Monitoramento e logs distribuídos
- Persistência de dados (stateful vs stateless)
- Curva de aprendizado para orquestração (Kubernetes)

**Boas práticas:**

- Criar imagens pequenas (minimizar camadas)
- Usar variáveis de ambiente para configuração
- Automatizar o build e o deploy
- Versionar o Dockerfile e o Compose no repositório

## 10. Resumo Conceitual

| <b>Conceito</b> | <b>Definição</b>                                  |
|-----------------|---------------------------------------------------|
| Contêiner       | Unidade isolada que executa uma aplicação         |
| Imagem          | Modelo imutável de um contêiner                   |
| Dockerfile      | Script que define como construir uma imagem       |
| Registry        | Local onde imagens são armazenadas                |
| Orquestração    | Coordenação automatizada de múltiplos contêineres |

## 11. Referências Bibliográficas

- MERNIK, M. *Software Architecture: Foundations, Theory, and Practice*. Wiley, 2010.
- RICHTER, J. *Architecture and Patterns for IT Service Management, Resource Planning, and Governance*. Springer, 2021.
- TURNBULL, J. *The Docker Book: Containerization is the New Virtualization*. 2020.
- Kubernetes Documentation: <https://kubernetes.io>
- Docker Documentation: <https://docs.docker.com>